

Questions & Answers

C++ Object-oriented Programming

AUTHOR

Khiem Nguyen

Lecturer in Multiscale Materials

khiem.nguyen@glasgow.ac.uk

1 Question 01

Just have a quick question about the mock UML diagram. In the solution you connect to an abstract class with a realisation. I'm thinking that this means we are treating abstract class as an interface. Are we only allowed to do this because we haven't provided an implementation for Draw(), and therefore the abstract class behaves like an interface? Or can we always use a realisation line with an abstract class, and it just forces subclass implementation regardless of any implementations we put into shape.

2 Answer 01

Programming design and thus concepts keeps evolving and at one point you might find my answer here is not fully correct. So, let us start:

Classically, the conceptual design is as follows:

- An **Abstract Class** defines an identity (*what it is*), so it needs data members to tract its state. The abstract class is defined by the fact that it has at least one pure virtual function. Conversely, we have **concrete class**. A child of the abstract class that implements all the pure virtual functions will become a **concrete class** and we can create an object of data type `ConcreteClass`. This concrete class implements or ""
 - For example, an Animal has pure virtual function `speak()` and has data members `weight` and `age`. Although I say "what an object is", we cannot create an object of data type `Animal`. That's why we give it the adjective **abstract**.
 - For example, the `Elephant` inherits `Animal`. `Elephant` is then an concrete class and we can create an object of class `Elephant`.
- An **Interface** defines a capability (*what it can do*). It is purely a behavioral contract and do not have internal state/internal data.

In C++, **there is no explicit `interface` keyword**.

- An "**Abstract Class** in C++" is a class that has at least one pure virtual function but is allowed to have member variables (data members) and concrete functions.
- An "**Interface** in C++" is literally just a class that contains **only pure virtual functions and no data members**.

Modern languages such as Java and C# may bend the rules, which I don't discuss here.

In UML Notation, we draw: - The **realization** represents where a concrete class implements an interface. This is visualized by a **dashed line** with a **hollow arrowhead** pointing toward the interface. - The **inheritance** represents "is-a" relationship between a more specific class (the subclass/child) and a broader, more general class (the superclass/parent), which including abstract classes. This is visualized by a **solid line** with a **hollow arrowhead** pointing toward the parent class (even if the parent class is an abstract class).

Coming back to Mock Exam

Let us come back to the UML class diagram question in the Mock exam. Whether `Shape` should be implemented as Abstract Class or Interface really depends on the designer. In our question, since `Shape` has zero data members in its description, it should be an Interface. It was my mistake that I did not mention this issue in the recorded review session. We don't have the keyword `interface` in C++ and thus I just wrote `<<abstract>>` although it should be `<<interface>>`. I have changed the solution from `<<abstract>>` to `<<interface>>` to reflect this error.

Then, do the students last year lose point because of this? **Answer: No.** If you look at my solution sheet (already uploaded on Moodle), you will see that I accept both ways: The students can make `Shape` as abstract class or interface, depending on the students. What's really important is that the students should make it clear: - If `Shape` is an interface, the student should draw "realization" line (dashed line with hollowed arrowhead). - If `Shape` is an abstract class, the student should draw "inheritance" line (solid line with hollowed arrowhead).

This is what I wrote in the solution sheet:

The students get partial marks expressing `Shape` as abstract class or interface. The students can implement `Shape` as abstract class or interface. They will have full grades for either abstract or interface. If the students picture `Shape` as abstract class, then `Ellipse` and `Polygon` inherit `Shape`. The students need to connect the inheritance arrows or implementation (realization) arrows between `Shape` and `Polygon`, and between `Shape` and `Ellipse` for full marks.

Important Note

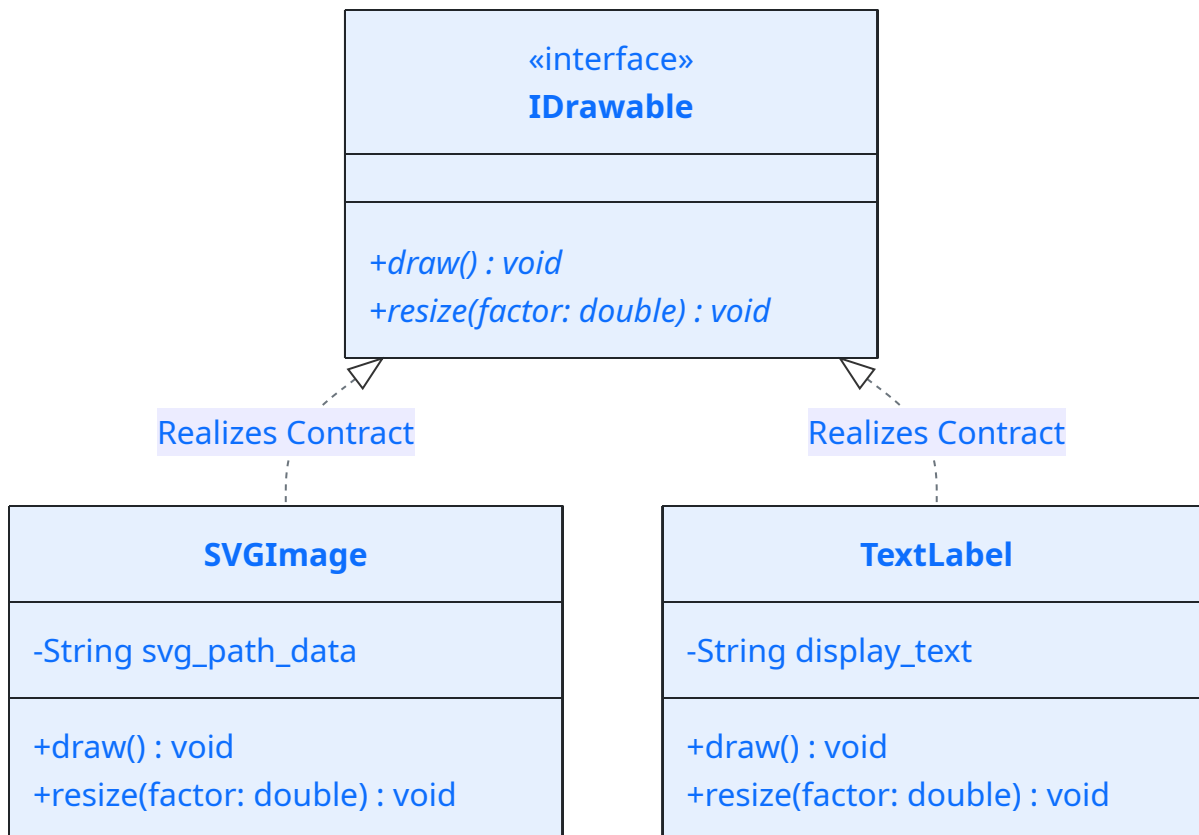
In reality, we may want to decide `Shape` as an abstract class and it fully depends on our program design. For example, if `Shape` has `color`, `centroid_position`, `line_thickness` as data members, it should be an abstract class. In our description, `Shape` does not have data members so it can be considered as an interface. If you ask Gemini or ChatGPT a general question (don't hint it what you want), it will likely give you two different approaches. I tried and it really explained two approaches: `Shape` can be either an interface or an abstract class, depending my functionality design.

What is really important to remember is that: - When a class has at least one pure virtual function and data members, it is an ****abstract class***. In the UML diagram, we use a solid line with hollowed arrowhead to visualize inheritance even if the parent class is an abstract class. - When a class has only

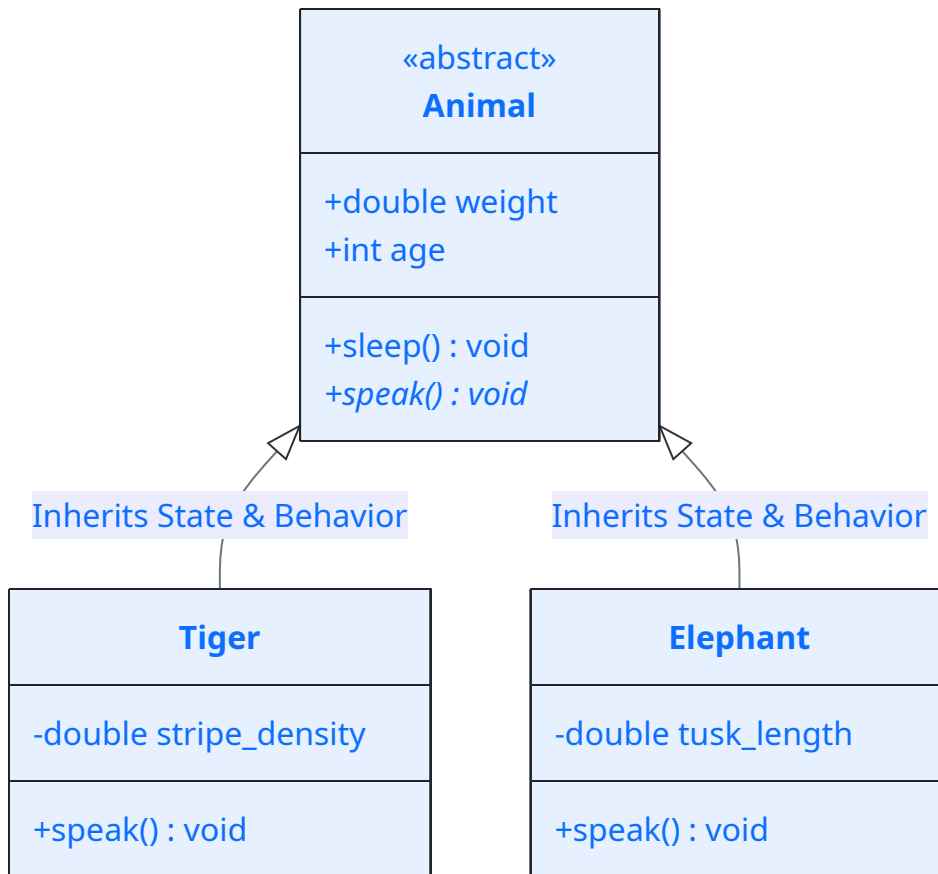
pure virtual functions and no data members, it is an **interface**. In the UML class diagram, we use a dashed line with hollowed arrowhead to visualize realization between the subclass and the interface class.

Examples: Note that you don't need to write text such as **Realizes Contract** and **Inherits State & Behavior** in the actual exam. The text here is just to illustrate the idea.

Interface



Abstract Class



How do you draw pure virtual function?

On paper, it is impossible to write in italic. In order to signify *italic* on paper, you write double asterisks `*` around the text you want to make.

`*+draw() : void*`